

- | | |
|----|--|
| 1. | Installation of Metamask and study spending Ether per transaction. |
|----|--|

<https://metamask.io/>

<https://chainlist.org/chain/11155111>

<https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

Practical-1: MetaMask Installation and Ether Spending Study

Step 1: Install MetaMask

For Desktop (Browser)

1. Open your preferred browser (Chrome, Firefox, Edge, or Brave).
2. Go to the official MetaMask website: <https://metamask.io>
3. Click "Download" and select your browser.
4. Add the extension to your browser.
5. Open MetaMask, and follow the setup:
 - Create a new wallet or import an existing one.
 - Set a strong password.
 - Backup your secret recovery phrase securely.

For Mobile

1. Go to your App Store (iOS) or Google Play Store (Android).
2. Search for MetaMask.
3. Download and install the app.
4. Follow similar setup steps (create wallet, backup phrase).

Step 2: Connect MetaMask to a Network

- By default, MetaMask uses the Ethereum Mainnet.
- To use test networks for learning, click the network dropdown and select a testnet like Goerli Test Network.
- Obtain free test Ether from a Goerli faucet: <https://goerlifaucet.com/>.

Step 3: Understanding Ether Spending per Transaction

Key Concepts

1. Gas Limit: Maximum units of gas you are willing to spend.
2. Gas Price: Price per gas unit in Gwei (1 ETH = 1,000,000,000 Gwei).
3. Transaction Fee = Gas Limit × Gas Price

Step 4: Make a Test Transaction

1. Ensure your wallet has ETH (test ETH on testnets).
2. Click Send in MetaMask.
3. Enter the recipient address and amount.
4. Adjust Gas Fee if desired.
5. Confirm the transaction.

Step 5: Observe Spending

- After sending, MetaMask displays the Transaction Fee in ETH.
- To see detailed info, check Etherscan:
 - i. Copy the transaction hash from MetaMask.
 - ii. Go to <https://etherscan.io> or testnet explorer like <https://goerli.etherscan.io>.
 - iii. Paste the hash to see gas used and ETH spent.

Step 6: Experiment

- Send multiple transactions with different gas prices.
- Observe how fees change based on network congestion.
- Try interacting with smart contracts to see higher gas usage.

Tips

- Always use a testnet first to avoid losing real ETH.
- MetaMask provides detailed transaction history and gas usage.

2.	Create your own wallet using Metamask for crypto transactions.
----	--

Practical-2: How to Create Your Own Wallet Using MetaMask for Crypto Transactions

MetaMask is a popular cryptocurrency wallet that allows you to manage Ethereum-based assets and interact with decentralized applications (dApps). Follow the steps below to create your own wallet.

Step 1: Install MetaMask

1. Go to the official MetaMask website: [<https://metamask.io>](https://metamask.io)
2. Click on **"Download"**.
3. Choose your platform:
 - **Browser Extension**: Chrome, Firefox, Brave, Edge
 - **Mobile App**: iOS or Android
4. Install the extension or app.

Step 2: Create a New Wallet

1. Open MetaMask.
2. Click on **"Get Started"**.
3. Select **"Create a Wallet"**.
4. Set a **strong password** for local access. This password is used to unlock your wallet on your device.
5. Click **"Create"**.

Step 3: Backup Your Secret Recovery Phrase

1. MetaMask will display a **12-word Secret Recovery Phrase**. This is extremely important.

2. Write it down on paper and **store it securely**.
- ⚠️ **Never share your recovery phrase with anyone.**
3. Confirm the recovery phrase by selecting the words in order.
4. Click **"Confirm"**.

Step 4: Access Your Wallet

1. Once confirmed, your wallet is ready.
2. You can view your wallet address by clicking on **Account 1** at the top.
3. This address can be shared to **receive crypto**.

Step 5: Add Funds to Your Wallet

1. Buy crypto from an exchange (like Coinbase, Binance, or WazirX).
2. Transfer it to your MetaMask wallet address.
3. Wait for the transaction to be confirmed.

Step 6: Use Your Wallet for Transactions

- **Send crypto:** Click **Send**, enter the recipient's address and amount.
- **Receive crypto:** Share your wallet address.
- **Interact with dApps:** Connect MetaMask to decentralized applications by clicking **Connect Wallet** on supported websites.

Step 7: Security Tips

- **Never share your password or Secret Recovery Phrase.**
- Enable **two-factor authentication (2FA)** on exchange accounts.
- Keep MetaMask updated to the latest version.
- Consider using **hardware wallets** for storing large amounts of crypto.

References

- [MetaMask Official Website](https://metamask.io)

- [MetaMask User Guide](https://metamask.zendesk.com/hc/en-us)

3.	Write a smart contract on a test network, for Bank account of a customer for following operations: • Deposit money • Withdraw Money • Show balance
----	--

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract BankAccount {
```

```
    mapping(address => uint256) private balances;
```

```
    event Deposit(address indexed account, uint256 amount);
```

```
    event Withdraw(address indexed account, uint256 amount);
```

```
    function deposit() public payable {
```

```
        require(msg.value > 0, "Deposit amount must be greater than 0");
```

```
        balances[msg.sender] += msg.value;
```

```
        emit Deposit(msg.sender, msg.value);
```

```
    }
```

```
    function withdraw(uint256 amount) public {
```

```
        require(amount > 0, "Withdraw amount must be greater than 0");
```

```
        require(balances[msg.sender] >= amount, "Insufficient balance");
```

```
        balances[msg.sender] -= amount;
```

```
        payable(msg.sender).transfer(amount);
```

```
        emit Withdraw(msg.sender, amount);
```

```
    }
```

```
    function getBalance() public view returns (uint256) {
```

```
        return balances[msg.sender];
```

```
    }
```

```
}
```

4.	Write a program in solidity to create Student data. Use the following constructs: • Structures • Arrays • Fallback Deploy this as smart contract on Ethereum and Observe the transaction fee and Gas values.
----	--

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract StudentData {
    struct Student {
        uint256 id;
        string name;
        uint8 age;
        string course;
    }

    Student[] private students;

    event StudentAdded(uint256 id, string name, string course);
    event FallbackCalled(address sender, uint value, string message);

    function addStudent(uint256 _id, string memory _name, uint8 _age, string memory _course)
    public {
        students.push(Student(_id, _name, _age, _course));
        emit StudentAdded(_id, _name, _course);
    }

    function getStudent(uint256 index) public view returns (uint256, string memory, uint8, string
memory) {
        require(index < students.length, "Invalid index");
        Student memory s = students[index];
        return (s.id, s.name, s.age, s.course);
    }

    function getTotalStudents() public view returns (uint256) {
        return students.length;
    }

    fallback() external payable {
        emit FallbackCalled(msg.sender, msg.value, "Fallback function triggered!");
    }
}
```

```
receive() external payable {  
    emit FallbackCalled(msg.sender, msg.value, "Receive function triggered!");  
}  
}
```

5.	Write a survey report on types of Blockchains and its real time use cases.
----	--